CS 4063 - Natural Language Processing

Due Date: Monday, April 20th by 11:55pm.

Assignments are to be done in groups of **two or three**. No late assignments will be accepted.
Submissions that do not comply with the specifications given in this document will not be marked and a zero grade will be assigned.

You are allowed to use all the generative tools that you have access to. But you should generate the minimal possible code to implement the reqired functionality. You should also be able to explain the code and produce the prompts when asked. **Be careful, this is not a one shot assignment and requires multiple pieces to be executed separately before it all come together. Learn!**

Each group must submit a **single GitHub repository link** on Google Classroom, containing all code, documentation, models, deployment scripts, and test cases. Also submit the vercel link that you have deployed your app on. **Do not submit your code, models, docker files on Google Classroom**. A short loom video to demo your system is optional but recommended.

RAG amd Tools in Conversational AI

Introduction

In the previous assignment, you designed and implemented a fully functional conversational AI system that ran entirely on a local CPU, exposed a real-time conversational API, and provided a minimal web-based chat interface. That system relied purely on prompt orchestration and conversational memory; tools and retrieval-augmented generation (RAG) were disallowed. You successfully built a low-latency, production-style system with a conversation manager, session handling, and a locally running quantized language model.

For this extension, you will take your existing voice chatbot and significantly enhance its capabilities by integrating RAG and external tools(with MCP). The goal is to transform your chatbot from a pure dialogue system into an intelligent assistant that can retrieve relevant information from a large document collection, manage customer relationships, and perform actions via multiple external tools. Additionally, you will optimise the system for real-time responses and optionally deploy it to the cloud.

This assignment emphasises system integration, tool orchestration, retrieval pipelines, and practical engineering decisions. You are expected to work in the same groups as before, and you must build upon your previous codebase. The final product will be a robust, extensible conversational agent that demonstrates modern techniques used in production LLM applications.

1 Overall Objective

The primary objective is to extend your existing chatbot to incorporate:

1. **Retrieval-Augmented Generation (RAG)** using a corpus of 50–100 documents. The chatbot must be able to answer user questions by retrieving and grounding its responses in those documents.
2. **A Customer Relationship Management (CRM) tool** that stores and retrieves information about users, their preferences, and interaction history.

3. **Three additional tools** of your choice (e.g., weather lookup, calendar scheduling, calculator, database query, email sender, etc.). These tools must be callable by the LLM during a conversation.
4. **Real-time response capabilities** — the system should stream responses with low latency, even when RAG or tools are invoked.
5. **Optional cloud deployment** using free-tier services. While not mandatory, successful deployment will be rewarded with extra credit.

All extensions must integrate seamlessly with your existing conversation manager, WebSocket API, and web frontend. The system should remain fully functional on a CPU (as before), but you are allowed to use lightweight vector databases and embedding models that run locally.

2 Detailed Requirements

The following subsections describe what you must implement. Unlike the previous point-wise description, here each requirement is explained in prose with specific technical expectations.

2.1 Retrieval-Augmented Generation (RAG)

Your system must be able to retrieve and use information from a collection of 50 to 100 documents. These documents can be of any type (e.g., PDF, plain text, markdown, HTML) but they should be relevant to your chosen business domain. For example, if your chatbot is for a dental clinic, the documents could include clinic policies, insurance information, treatment descriptions, and frequently asked questions.

You are free to choose any local embedding model (e.g., "all-MiniLM-L6-v2" via Sentence-Transformers, or any GGUF-compatible embedding model) and any vector store that runs on CPU (e.g., Chroma, FAISS, or a simple in-memory store). When a user asks a question, the system must:

- Embed the user's query and retrieve the top-k most relevant document chunks (k is your choice, but should be at least 3).
- Inject the retrieved chunks into the LLM prompt in a way that does not exceed the model's context window. You must implement a context management strategy (e.g., truncating older conversation turns or summarising retrieved passages).
- Ensure that the LLM's response is grounded in the retrieved content. The chatbot should be able to cite or refer to the documents when appropriate.

You are expected to build an offline indexing pipeline that processes the 50–100 documents, chunks them (e.g., 512 tokens per chunk with overlap), computes embeddings, and stores them in the vector database. The pipeline should be re-runnable so that documents can be updated.

2.2 Customer Relationship Management (CRM) Tool

The CRM tool is a mandatory component that must be implemented as a callable tool (see Section 2.3). Its purpose is to remember information about individual users across sessions and to provide personalised responses. At a minimum, the CRM tool must support the following operations:

- Store user information (e.g., name, contact details, preferences, past interactions) keyed by a unique session ID or user ID.
- Retrieve user information when a conversation starts, so that the chatbot can greet the user by name or recall previous requests.
- Update user information during a conversation (e.g., the user provides a new phone number or changes an appointment preference).

You may implement the CRM as a simple key-value store (e.g., a Python dictionary with persistence to a JSON file or SQLite), or as a more sophisticated database. The CRM must be exposed as a tool that the LLM can decide to call. For example, the LLM might call `get_user_info(user_id)` when a returning user says "I'm back", or `update_user_info(user_id, field, value)` when the user provides new details.

2.3 Three Additional Tools

Beyond the CRM tool, you must implement three other tools that perform useful actions. These tools can be entirely local or call external APIs (as long as you respect the free tier limits). Examples of suitable tools include:

- A weather tool that fetches current weather for a given city using a free API (e.g., OpenWeatherMap).
- A calendar tool that reads and writes events to a local calendar file or Google Calendar API.
- A calculator tool that evaluates mathematical expressions.
- A stock price tool that retrieves live or delayed stock prices.
- A news summarisation tool that pulls headlines from RSS feeds.
- A database query tool that searches a product catalogue or a local SQLite database.

Each tool must be defined with a clear name, description, and a JSON schema for its arguments. Your LLM orchestration layer must be capable of detecting when a tool should be invoked, calling the tool with the extracted arguments, and returning the tool's output back to the LLM to formulate a natural language response. You may implement tool calling using prompt-based function calling (e.g., instructing the LLM to output a special JSON structure) or by using libraries like LangChain or LlamaIndex. However, you are expected to understand the underlying mechanism.

All tools must be executed asynchronously so that they do not block the main conversation thread. Timeout and error handling are required: if a tool fails (e.g., network error, invalid input), the chatbot should gracefully inform the user.

2.4 Real-Time Responses

Real-time interaction is a core requirement of the original assignment and must be preserved and even improved. The addition of RAG and tools introduces potential latency (embedding queries, vector search, tool API calls). You must implement strategies to keep the end-to-end response time low. Specifically:

- Token streaming must continue to work from the WebSocket endpoint. Users should see the response appearing word by word.
- RAG retrieval and tool calls should be performed before the LLM starts generating (or, in advanced designs, interleaved). However, any pre-processing must not cause noticeable delays. You should benchmark retrieval and tool call times and aim to keep them under 2 seconds combined.
- Use caching where appropriate: e.g., cache embeddings for repeated queries, or cache tool results that are unlikely to change rapidly.
- The system must support concurrent users as before. When multiple users trigger RAG or tools simultaneously, the system should remain responsive.

You will be evaluated on the perceived responsiveness of the chatbot in a live demo.

2.5 Cloud Deployment (Optional)

Cloud deployment is optional but encouraged. If you choose to deploy your extended chatbot, you must use a free-tier service such as:

- **Render** (free tier with web services),
- **Hugging Face Spaces** (with Gradio or Streamlit, though WebSocket support may be limited),
- **Oracle Cloud Free Tier** (always-free ARM instances),
- **Google Cloud Run** (free monthly quota),
- **AWS Free Tier** (e.g., EC2 t2.micro).

Because the LLM and RAG components are computationally heavy, you may need to keep the core inference local and only deploy the API and frontend, or you may use a very small quantised model that fits in the free tier's memory. A successful deployment means that the entire system (including LLM, RAG, and tools) runs in the cloud and is accessible via a public URL. You must document any trade-offs you made (e.g., using an even smaller model, reducing document count). Extra credit will be awarded based on the completeness and stability of the cloud deployment.

3 System Architecture

Your existing architecture consists of a web frontend, a FastAPI backend with WebSocket, a conversation manager, and a local LLM engine. For this extension, you will augment the conversation manager with two new modules: a **retrieval module** (embedding + vector store) and a **tool orchestrator** (handles tool registration, argument parsing, and execution). The figure below illustrates the high-level data flow (you are expected to produce a similar diagram in your report).

```
[Web UI] <--WebSocket--> [FastAPI + WebSocket] <--> [Conversation Manager]
                                                          |
                                                          +--> [Retrieval Module]
                                                          |       (Embedding + Vector DB)
                                                          +--> [Tool Orchestrator]
                                                                  |
                                                                  +--> [CRM Tool]
                                                                  +--> [Tool 1]
                                                                  +--> [Tool 2]
                                                                  +--> [Tool 3]
                                                          |
                                                          +--> [LLM Engine]
```

The conversation manager receives a user message, decides whether to retrieve relevant documents (RAG) and/or invoke a tool, then constructs a prompt that includes conversation history, retrieved context, and tool results. The LLM generates a response, which may contain a tool call request. The orchestrator executes the tool, feeds the result back to the LLM for a final answer, and streams the answer to the frontend.

You are free to modify the existing conversation manager as needed, but you must maintain the same API contract (WebSocket JSON messages) so that the frontend continues to work without changes.

4 Implementation Guidelines

You are expected to write clean, well-documented Python code. Use asynchronous programming (asyncio) throughout to handle concurrency. The following libraries are recommended but not mandatory:

- `llama-cpp-python` or `ctransformers` for local LLM inference.
- `sentence-transformers` for embedding generation.
- `chromadb` or `faiss-cpu` for vector storage.
- `pydantic` for data validation and tool schemas.
- `httpx` for asynchronous API calls to external tools.

Your code must be containerised with Docker as before. The Docker image should include all dependencies and be able to start the FastAPI server and the LLM engine (if they are separate processes, use a multi-container setup with docker-compose). Provide clear instructions for building and running the system.

5 Evaluation Criteria

Your work will be assessed based on the following criteria:

- **Correctness and completeness** (40%): All required components (RAG with 50–100 documents, CRM tool, three additional tools, real-time streaming) are present and work correctly. The RAG retrieval returns relevant chunks, and the LLM uses them appropriately. The CRM tool correctly stores and retrieves user data. Each of the three additional tools performs its intended function.
- **System integration and orchestration** (20%): The conversation manager seamlessly switches between RAG, tool calling, and plain dialogue. Tool calls are properly parsed and executed; the LLM receives tool outputs and generates coherent answers. Error handling is robust.
- **Performance and real-time behaviour** (20%): Response latency is low (measured and reported). Streaming works without hiccups. The system handles at least two concurrent users (simulated) without significant degradation.
- **Documentation and reproducibility** (20%): The README.md provides a detailed architecture diagram, setup instructions, model choices, performance benchmarks, and known limitations. The code is well organised and commented. The submission includes a Postman collection and a working Docker setup.
- **Extra credit** (up to 10%): Successful cloud deployment with public access; innovative tools or advanced RAG techniques (e.g., hybrid search, re-ranking); comprehensive stress test results.

6 Deliverables

Each group must submit a single archive (or a link to a private GitHub repository) containing the following:

1. All source code for the extended chatbot, including the frontend (HTML/JS) and backend.
2. A `docker-compose.yml` file (or `Dockerfile`) that builds and runs the entire system.
3. A `requirements.txt` or `pyproject.toml` listing all dependencies.
4. The document collection (50–100 documents) used for RAG, either as part of the repository or with a script to download them.
5. A Postman collection that tests all key endpoints (WebSocket chat, and optionally any REST endpoints you added).
6. A comprehensive `README.md` following the structure described in Section 6.1.
7. A short video demo (3–5 minutes) showing the chatbot in action: demonstrate a conversation that uses RAG, a conversation that invokes the CRM tool, and another that uses at least two of the three additional tools. Upload the video to a streaming platform (unlisted YouTube) and include the link in the README.

6.1 README.md Structure

The README must contain the following sections, each written in clear prose:

- **Project title and group members.**
- **Business use case**: Describe the domain of your chatbot and explain how RAG and tools add value. *Note: You can change the business use-case to make it more interesting!*
- **Architecture diagram** (image) and a textual explanation of each component.
- **Model selection**: Which quantised LLM you used, why, and its performance characteristics (tokens per second, memory usage).

- **Document collection**: Number of documents, their source, how they were chunked, embedding model, vector database, and retrieval parameters (top-k, similarity metric).
- **Tools description**: For each of the four tools (CRM + three others), provide the tool's name, description, input schema, and an example of the LLM invoking it.
- **Real-time optimisation**: Explain what you did to minimise latency (caching, asynchronous retrieval, etc.). Provide benchmark numbers: average retrieval time, tool call latency, and end-to-end response time for a typical query.
- **Setup instructions**: Step-by-step commands to clone, build the Docker image, and run the system. Include any environment variables or configuration files.
- **Known limitations**: What does your system not handle well? (e.g., very long documents, certain tool edge cases, high concurrency.)
- **Cloud deployment (if attempted)**: URL, free tier used, any modifications made for cloud operation, and observed performance.

7 Submission Guidelines

- Submit a `.zip` file or a link to a GitHub repository. The submission must be uploaded to GC.
- The video demo link must be accessible (no login required).

Honor Policy

Cheating in code, datasets, or reports will result in an immediate **zero grade** and reporting to the academic disciplinary committee. All work must be original to the group.